

- ***Cursor***
- ***Administration Tasks***
 - ***Backup***
 - ***Restore***
- ***SQLCLR***

Additional Topics
Linked servers
Service broker
Securing SQL server and DB encryption
Full-text search
SQL injection

Cursor

- 📖 A cursor is a mechanism that processes the result set of a predefined query on a row-by-row basis

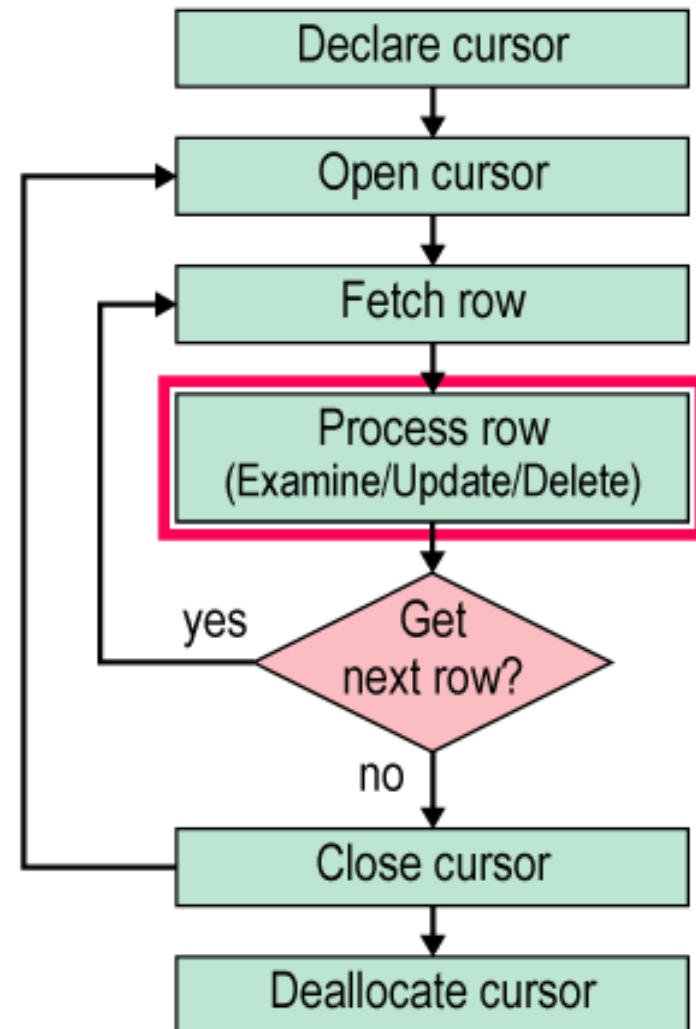
Understanding Cursors

Cursors extend processing of result sets

- Allow positioning at specific rows
- Retrieve one or more rows from the current position
- Support data modification
- Support different levels of visibility
- Provide T-SQL statements access to data

Cursor Life Cycle

1. Declare the cursor
2. Open the cursor
3. Fetch each row
4. Close the cursor
5. Deallocate the cursor



Declare the Cursor

 When you declare a cursor:

 You specify the query

 You specify the cursor mode

● **for read only**

● **for update**

 Adaptive Server reserves memory resources for the cursor

 Update Cursors is the Default Mode

Global Variables for Cursors

@@rowcount

-  After a **fetch**, returns the total number of rows returned by all **fetch** statements for that cursor
-  After any DML statement in a cursor, returns the number of rows affected by that statement

@@sqlstatus

-  Returns 0 if the previous **fetch** was successful
-  Returns 1 for an error resulting from a **fetch**
-  Returns 2 if the previous **fetch** failed because there are no more rows to **fetch**

Alternatives to Cursors

- 📖 Cursors are not always the only way to accomplish a given task

- 📖 Cursor Code Example 1 rewritten using the **case** statement:

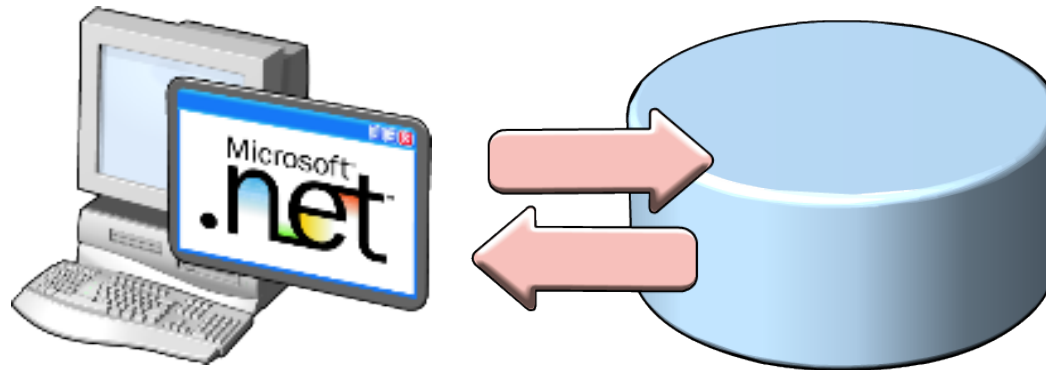
```
select title_id, type,  
       case type  
         when "business" then price * $1.08  
         when "mod_cook" then price  
       end  
from titles  
where type in ("business", "mod_cook")
```

- 📖 Cursor Code Example 1 rewritten using two queries:

```
select title_id, type, price * $1.08  
from titles  
where type = "business"  
select title_id, type, price  
from titles  
where type = "mod_cook"
```

CLR Integration , Assemblies

- Introduced in SQL Server 2005
- Deploy custom developed CLR assemblies within the SQL Server Process
- Leverage the .NET 2.0 API



Implementing Custom Aggregate Functions

- Custom programs in an assembly that are imported into SQL Server® using the integrated CLR
- Can be created using any .NET language such as C# and VB

```
CREATE ASSEMBLY StringUtilities FROM  
'PathToAssembly\StringUtilities.dll'
```

```
WITH PERMISSION_SET=SAFE;
```

```
CREATE AGGREGATE Concatenate(@input nvarchar(4000))  
RETURNS nvarchar(4000)  
EXTERNAL NAME [StringUtilities].[Concatenate]
```